II. If no transaction has expired or `transactionTimestamp` for the expired transactions has been dropped, then we add the new `transactionTimestamp` to `transactionList` and write out the customer record.

Now that we have outlined the goal of our transformation, let's look at the code. To follow along in your own workspace, please refer to the following notebooks:

- `CH5-01-Generating Records`
- `CH5-02-Auto Loader`
- `CH5-03-FE Using Spark Structured Streaming`
- `CH5-04-Building Maximum Price Feature Table`

Before you begin executing *notebook 3*, please open *notebooks 1* and *2* and click **Run All** for both. These two notebooks relaunch the data streams that we need running in order to run *notebook 3*. However, you do not need all streams running in order to run *notebook 4*.

The code we are jumping into is in the `CH5-03-FE Using Spark Structured Streaming` notebook. We start with the basics – imports and delta configurations – as shown in the following screenshot:

```
import java.time.Instant
import java.util.concurrent.TimeUnit
import scala.collection.mutable.ListBuffer
import org.apache.spark.sql.streaming.{GroupStateTimeout, OutputMode, GroupState}

spark.conf.set("spark.databricks.delta.optimizeWrite.enabled", "true")
spark.conf.set("spark.databricks.delta.autoCompact.enabled", "true")
```

Figure 5.10 – Setting delta configurations for optimized writes and compaction

These configurations can also be set in the cluster configurations, but we call them out explicitly in *Figure 5.10*. These settings will automatically compact sets of small files into larger files as it writes for optimal read performance. *Figure 5.11* primarily shows the reset commands that enable starting fresh when needed:

```
%python
table_name = "transaction_count_ft"
history_table_name = "transaction_count_history"
if bool(dbutils.widgets.get('Reset')):
  dbutils.fs.rm(f"{volume_file_path}/{table_name}/streaming_outputs/", True)
  dbutils.fs.rm(f"{volume_file_path}/{history_table_name}/streaming_outputs/", True)
  sql(f"DROP TABLE IF EXISTS {table_name}")
  sql(f"DROP TABLE IF EXISTS {history_table_name}")
```

Figure 5.11 – Setting the widget value to True runs the commands in this cell, removing the output data, including the checkpoint, and dropping the table